

A Prototype of an Intelligent Chatbot Application Integrated with OpenLike LLM Models

Armando Soares Sousa
Federal University of Piauí – UFPI
Teresina, Piauí, Brazil
armando@ufpi.edu.br

Abstract

Large Language Models (LLMs) have accelerated the development of intelligent conversational applications in education, technical support, decision-making, software engineering, and process automation. However, building a complete LLM-based chatbot requires the integration of multiple software components, including an inference server, an intermediary application programming interface (API), authentication mechanisms, conversational history persistence, and a user-facing web interface. This paper presents a prototype of an intelligent chatbot application integrated with LLMs through an OpenAI-compatible API pattern, referred to in this work as an *OpenLike* architecture. The analyzed solution is organized into two independent but integrated projects: a backend implemented with FastAPI and SQLAlchemy, responsible for authentication, user management, chat sessions, message persistence, and communication with an LLM server; and a frontend implemented with Flask, responsible for user interaction, login and registration screens, dashboard views, chat rendering, and synchronous or streaming prompt submission. The paper describes the solution architecture, the internal structure of the backend and frontend projects, the data model, the authentication flow, the integration with the LLM server, the streaming mechanism, and the local execution procedure. We also discuss technical limitations, security concerns, and possible evolution paths, including Retrieval-Augmented Generation (RAG), observability, containerization, multi-model support, and production-oriented deployment practices.

CCS Concepts

- **Computing methodologies** → **Natural language generation**;
- **Information systems** → **Web applications**;
- **Software and its engineering** → **Software architectures**;
- **Security and privacy** → *Authentication*.

Keywords

intelligent chatbot, large language models, LLM, FastAPI, Flask, OpenAI-like API, OpenLike architecture, generative AI, streaming, web applications

1 Introduction

The popularization of Large Language Models (LLMs) has significantly changed how interactive software systems are designed. Applications that previously relied on rigid question-answer flows can now incorporate text generation, contextual reasoning, summarization, classification, translation, code

explanation, and decision support. As a result, modern chatbots have become more flexible, adaptive, and capable of interacting with users through natural language.

A major technical foundation of contemporary LLMs is the Transformer architecture, introduced by Vaswani et al. in *Attention Is All You Need* [8]. By replacing recurrence and convolution with attention mechanisms, Transformers enabled more parallelizable sequence modeling and became a core architectural pattern behind many modern LLM families. More recent model families, such as Qwen3, extend this line of work with dense and Mixture-of-Experts variants, multilingual capabilities, and reasoning-oriented operation modes [10].

Nevertheless, building a complete application on top of an LLM requires more than selecting a model. It also requires an architecture that integrates a user interface, backend services, authentication, authorization, persistent storage, inference endpoints, and user experience mechanisms such as incremental response streaming. In real applications, the LLM server is often not exposed directly to end users. Instead, an intermediary backend acts as a control layer responsible for security, auditing, data persistence, API normalization, usage governance, and integration with external or local inference providers.

This paper presents a prototype of an intelligent chatbot application integrated with LLMs through an OpenAI-compatible API pattern. We refer to this type of integration as an *OpenLike* architecture because the application expects an inference server exposing endpoints conceptually aligned with common OpenAI-style interfaces, such as `/v1/models` and `/v1/chat/completions`. The solution is composed of two projects: a backend implemented with FastAPI and a frontend implemented with Flask. The backend provides JSON Web Token (JWT)-based authentication, user management, chat sessions, message persistence, health checks, model listing, and LLM inference. The frontend provides registration, login, dashboard, chat, history visualization, and proxy routes for standard and streaming inference.

The main contribution of this paper is a technical characterization of the prototype architecture and its implementation decisions. Specifically, the paper describes: (i) the architectural decomposition of the chatbot solution; (ii) the backend structure, data model, authentication flow, and inference API; (iii) the frontend structure and interaction flow; (iv) the synchronous and streaming communication mechanisms; (v) local execution procedures; and (vi) limitations and future

directions for evolving the prototype into a more robust LLM application platform.

2 Background

2.1 Large Language Models and Conversational Applications

LLMs are deep learning models trained on large-scale text corpora to model linguistic, semantic, and contextual patterns. They can generate coherent responses to natural language instructions, which makes them suitable for intelligent assistants, educational systems, technical support, software engineering tools, and knowledge-intensive workflows. Transformer-based models rely on attention mechanisms to represent relationships among tokens in a sequence [8]. This design supports scalable training and has become a foundational mechanism for generative language models.

The analyzed prototype is configured to use a Qwen-family model identifier by default. Qwen3 is described as a family of LLMs designed to improve performance, efficiency, reasoning, and multilingual capabilities, including dense and Mixture-of-Experts architectures [10]. Although the prototype does not depend conceptually on a specific LLM family, the default configuration indicates a local or self-hosted inference scenario.

2.2 OpenAI-Compatible and OpenLike APIs

A practical trend in LLM application development is the adoption of API shapes compatible with widely used OpenAI endpoints. This strategy allows developers to use existing client libraries while pointing them to alternative inference servers through a configurable base URL. The official OpenAI Python library supports client configuration with custom base URLs, which is useful when integrating with OpenAI-compatible services [4]. In this paper, the term *OpenLike* denotes this architectural style: the application integrates with an LLM server that behaves like an OpenAI-compatible backend, without requiring the server to be the OpenAI cloud service itself.

2.3 Backend and Frontend Frameworks

FastAPI is a modern Python framework for building APIs with type hints, data validation, and automatically generated OpenAPI documentation [1]. These characteristics make it suitable for building backend services that expose structured REST endpoints and validate request and response payloads.

Flask is a lightweight WSGI web application framework designed to support quick development while allowing applications to scale in complexity [6]. In the prototype, Flask is used as the frontend web application layer, rendering templates and forwarding user actions to the backend.

2.4 Streaming with Server-Sent Events

LLM responses may take time to generate, especially when outputs are long or when inference runs on local hardware.

Table 1: High-level summary of the analyzed projects.

Aspect	Backend	Frontend
Main framework	FastAPI	Flask
Primary role	API, authentication, persistence, LLM integration	Web interface, templates, chat UI, API proxy
Execution port	8000	5001
Persistence	SQLAlchemy ORM	Browser/session plus backend storage
Authentication handling	JWT validation and issuing	Stores backend token in Flask session
LLM interaction	OpenAI-compatible async client	Calls backend proxy routes
Streaming	Async generator and HTTP streaming	Fetch stream reader and incremental rendering

Streaming mitigates perceived latency by progressively delivering generated tokens to the browser. Server-Sent Events (SSE) define a browser-supported mechanism for server-to-client event streams over HTTP. MDN describes event streams as UTF-8 text streams with messages separated by pairs of newline characters [3]; the WHATWG HTML Living Standard specifies the `EventSource` interface for receiving such server-sent events [9]. The prototype follows an SSE-like format by sending chunks as `data: {...}` messages.

3 Methodology

This work adopts a technical inspection methodology. The analyzed artifacts consisted of two compressed project archives: a backend project named `mlx-openai-like-backend-main.zip` and a frontend project named `mlx-openai-like-frontend-main.zip`. The inspection considered project structure, dependency files, source modules, configuration variables, route definitions, database models, schemas, templates, JavaScript logic, and execution instructions.

The backend project contains 29 files and approximately 1,299 lines, including lock files and source code. The frontend project contains 16 files and approximately 2,082 lines, including templates, CSS, JavaScript, and application code. Table 1 summarizes the high-level characteristics of the two projects.

4 Solution Overview

The solution is organized as a three-layer architecture: (i) the browser-based user interface, served by Flask; (ii) the FastAPI backend, responsible for application logic and persistence; and (iii) an OpenLike LLM inference server, expected to

expose OpenAI-compatible endpoints. Figure 1 presents the conceptual architecture.

This separation of concerns has two important advantages. First, the frontend does not need to know details about the inference server. Second, the backend centralizes security, data ownership checks, persistence, and future governance features such as logging, rate limiting, moderation, or cost tracking.

5 Backend Design

5.1 Project Structure

The backend follows a modular FastAPI structure:

```
mlx-openai-like-backend-main/
|-- pyproject.toml
|-- uv.lock
|-- main.py
`-- app/
    |-- main.py
    |-- core/
    |   |-- config.py
    |   |-- database.py
    |   |-- security.py
    |-- models/
    |   |-- user.py
    |   |-- chat.py
    |-- routers/
    |   |-- auth.py
    |   |-- chats.py
    |   |-- deps.py
    |   |-- health.py
    |   |-- inference.py
    |   |-- models.py
    |-- schemas/
    |   |-- auth.py
    |   |-- chat.py
    |   |-- mlx.py
    |   |-- models.py
    `-- services/
        |-- mlx_client.py
```

Listing 1: Simplified backend structure.

The **core** package concentrates infrastructure configuration, database connection, and security functions. The **models** package defines SQLAlchemy ORM entities. The **schemas** package defines Pydantic models for request and response validation. The **routers** package exposes REST endpoints. The **services** package encapsulates communication with the LLM server.

5.2 Dependencies

The backend project declares dependencies that can be grouped into API, validation, persistence, security, and LLM integration categories. FastAPI and Uvicorn implement the ASGI API layer. Pydantic and Pydantic Settings support type-oriented validation and configuration. SQLAlchemy provides the object-relational mapper. Passlib, bcrypt, and python-jose support password hashing and JWT operations. The OpenAI Python client and HTTPX support asynchronous integration with the OpenLike inference service.

5.3 Configuration

The backend is configured mainly through environment variables. The default LLM server base URL is built from the `LLM_SERVER_PORT` variable, resulting in <http://localhost:8080/v1>. The default model identifier is `mlx-community/Qwen3-4B-Instruct-2507-4bit`. The backend also defines variables for

Table 2: Backend dependency categories.

Category	Dependencies
API service	fastapi, uvicorn
Validation and configuration	pydantic, pydantic-settings, python-dotenv
Persistence	sqlalchemy
Security	passlib, bcrypt, python-jose
LLM integration	openai, httpx

database access, JWT secret, JWT algorithm, token expiration, and CORS origins.

A technical inconsistency was identified in the default database configuration. The default `DATABASE_URL` points to PostgreSQL, while the SQLAlchemy engine is configured with `connect_args={"check_same_thread": False}`, which is specific to SQLite. For local execution, it is therefore recommended to explicitly use SQLite:

```
DATABASE_URL=sqlite:///./mlx_chat.db
```

Listing 2: Recommended local database configuration.

For production, a PostgreSQL driver should be added and the SQLite-specific connection argument should be conditionally applied only when the database URL uses SQLite.

5.4 Application Entry Point

The backend creates a FastAPI application, configures Cross-Origin Resource Sharing (CORS), initializes database tables at startup, and includes routers for authentication, chats, inference, health checks, and model listing. FastAPI automatically exposes interactive documentation through OpenAPI-compatible interfaces such as Swagger UI [1].

5.5 Data Model

The data model is composed of three main entities: **User**, **ChatSession**, and **ChatMessage**. SQLAlchemy’s declarative mapping style is appropriate for this kind of object-relational modeling [7]. Figure 2 shows the conceptual entity relationship model.

A **User** can own many chat sessions. A **ChatSession** belongs to a user and can contain many messages. A **ChatMessage** stores the role and textual content of a message. The supported roles follow the common conversational model: **user**, **assistant**, and potentially **system**.

5.6 Authentication and Authorization

The authentication flow uses JWT. During registration, a user provides an email and password. The backend checks if the email already exists, hashes the password using bcrypt through Passlib, and stores the resulting password hash. During login, the backend validates the credentials and returns a bearer token. Protected endpoints depend on a function that

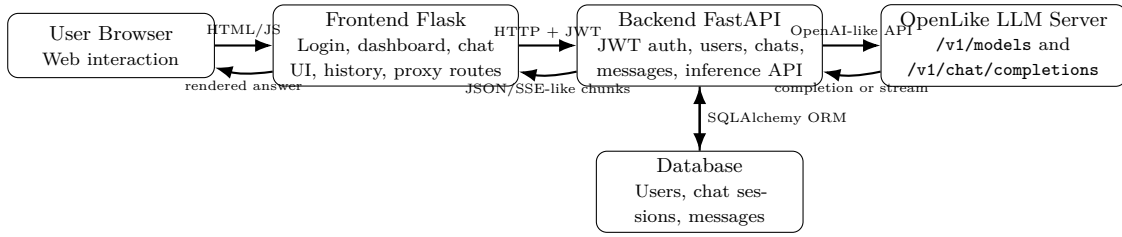


Figure 1: Conceptual architecture of the OpenLike chatbot prototype.

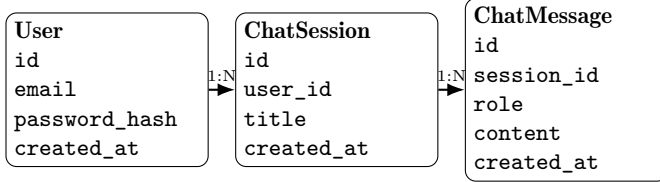


Figure 2: Conceptual data model for users, sessions, and messages.

extracts the bearer token, decodes it, retrieves the subject, and loads the corresponding user from the database.

OWASP notes that JWTs are commonly used as authentication or session tokens in REST APIs and that vulnerabilities in their implementation may compromise an application [5]. In this prototype, production deployment should replace default secrets, avoid logging tokens, enforce HTTPS, define token expiration policies, and consider refresh-token rotation where applicable.

5.7 Chat Management Endpoints

The backend exposes three main chat endpoints: `POST /chats`, `GET /chats`, and `GET /chats/{chat_id}/messages`. All of them require authentication. The implementation verifies ownership before returning messages from a chat session. This is an important access-control step, since chat histories may contain sensitive user data.

5.8 LLM Client Service

The integration with the OpenLike LLM server is encapsulated in `services/mlx_client.py`. The service creates an asynchronous OpenAI-compatible client using a configurable `base_url`. This design allows the backend to communicate with a local or remote inference server that implements compatible endpoints.

The core request sends a system message and the user prompt to the chat completion endpoint. The system message instructs the model to behave as a helpful and concise assistant. The method supports both standard completion and streaming mode. This design allows the same backend endpoint to support two user experience styles.

A relevant improvement is to ensure that the model selected by the frontend is passed to the LLM client. In the current design, the default model from settings may override

the model field sent by the frontend. A more flexible implementation would accept a model parameter and fall back to the configured default only when no model is provided.

5.9 Inference Endpoint

The central backend endpoint is `POST /generate`. It receives a model identifier, a prompt, a chat identifier, a streaming flag, and optional parameters. Its execution flow is shown in Figure 3.

The endpoint first validates the authenticated user and verifies whether the requested chat belongs to that user. It then persists the user message, invokes the LLM service, receives either a full response or a stream of chunks, persists the assistant response, and returns the result to the frontend.

5.10 Streaming Behavior

In streaming mode, the backend accumulates generated tokens and yields partial chunks. The emitted format resembles SSE:

```
data: {"type": "token", "chunk": "partial text"}\n\n
```

Listing 3: SSE-like chunk format emitted by the backend.

However, the prototype uses `application/x-ndjson` as the media type while the payload follows an SSE-like `data: ...` structure. Since SSE commonly uses `text/event-stream`, the backend should align the media type with the payload format. This would improve compatibility with browser APIs and developer expectations [3, 9].

6 Frontend Design

6.1 Project Structure

The frontend is implemented as a Flask application with HTML templates, CSS, and JavaScript:

```
mlx-openai-like-frontend-main/
|-- pyproject.toml
|-- uv.lock
|-- main.py
-- app/
  |-- main.py
  |-- static/
  | |-- css/styles.css
  | |-- js/chat.js
  -- templates/
    |-- base.html
    |-- register.html
    |-- login.html
    |-- dashboard.html
    |-- chat.html
    -- chat_history.html
```

Listing 4: Simplified frontend structure.

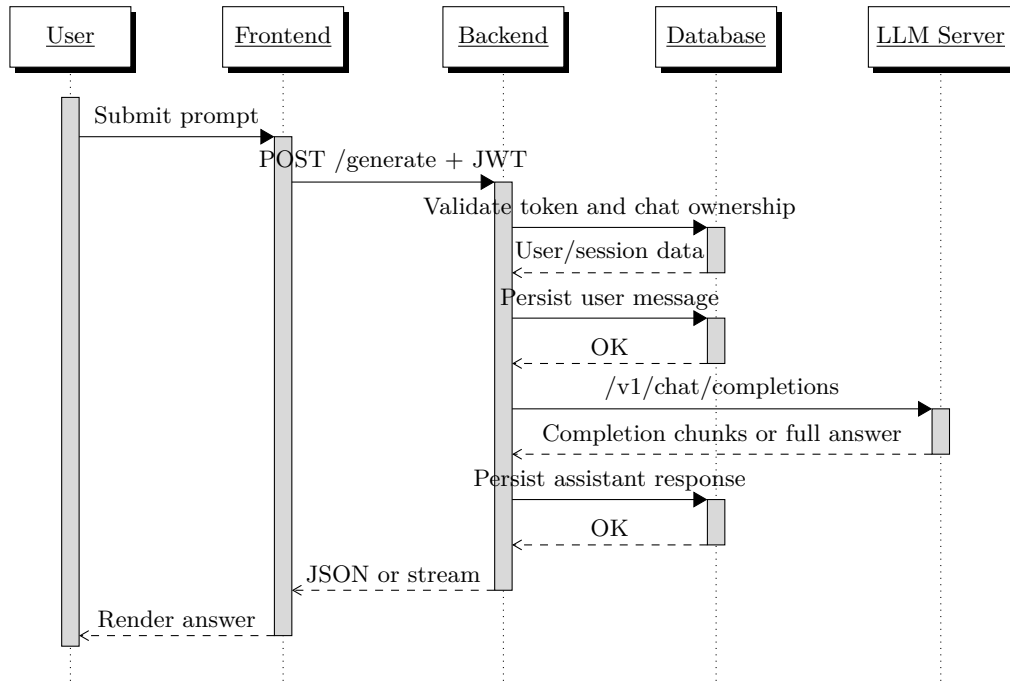


Figure 3: Simplified inference sequence across frontend, backend, database, and LLM server.

The application uses server-side rendering through Jinja templates. Bootstrap and icon libraries are included in the templates to support layout and visual elements. Custom CSS defines the appearance of the chat interface and dashboard. JavaScript handles asynchronous message submission and streaming rendering.

6.2 Frontend Dependencies

The frontend declares only two main dependencies: **flask** and **requests**. Flask provides the web framework and templating infrastructure, while **requests** allows the frontend server to communicate with the backend API. The simplicity of this dependency set makes the frontend easy to execute and suitable for instructional or prototyping scenarios.

6.3 Routes and Views

Table 3 summarizes the main frontend routes.

The frontend stores the backend JWT access token in the Flask session after successful login. It then uses this token to add an **Authorization: Bearer <token>** header when calling protected backend endpoints.

6.4 Chat Interface

The chat interface includes a prompt input area, a model selector, a streaming toggle, a message display area, and status indicators. The JavaScript file **chat.js** implements the dynamic behavior. It adds user and assistant messages to the interface, formats code blocks, calls the frontend proxy

Table 3: Main frontend routes.

Route	Purpose
GET /	Redirects users to login or dashboard
GET /register	Shows the registration form
POST /register	Sends registration data to the backend
GET /login	Shows the login form
POST /login	Authenticates the user through the backend
GET /dashboard	Lists chats and service status
POST /chats/new	Creates a new chat session
GET /chat/<id>	Opens the chat interface
GET /chat/<id>/history	Shows persisted chat history
POST /api/generate	Proxy route for non-streaming inference
POST /api/stream	Proxy route for streaming inference

endpoints, reads streaming chunks, and updates the displayed assistant answer incrementally.

6.5 Streaming on the Client Side

For streaming, the frontend uses the Fetch API and reads the response body as a stream. The client decodes chunks with **TextDecoder**, splits incoming data by line, parses lines that

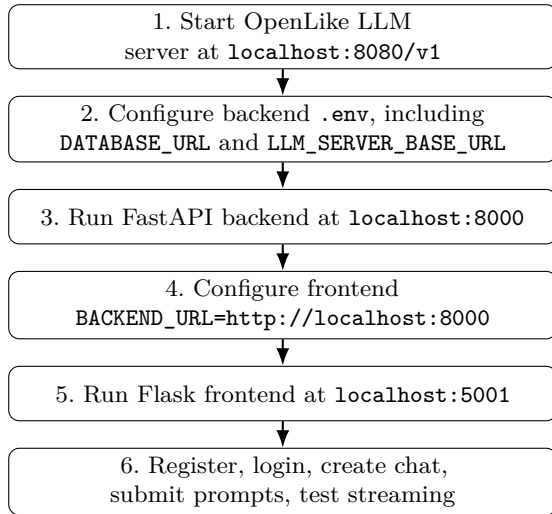


Figure 4: Execution pipeline for the complete chatbot solution.

start with `data:`, and appends token fragments to the visible assistant message. This design gives the user immediate feedback while the model continues generating the response.

7 Integrated Execution Procedure

The complete solution requires three processes: the OpenLike LLM server, the FastAPI backend, and the Flask frontend. The expected default ports are 8080 for the LLM server, 8000 for the backend, and 5001 for the frontend. Figure 4 summarizes the execution pipeline.

7.1 Backend Execution

A local backend configuration can be defined as follows:

```

APP_NAME="MLX OpenAI Like Backend"
DEBUG=True
LLM_SERVER_PORT=8080
LLM_SERVER_BASE_URL=http://localhost:8080/v1
MODEL_O_LLM=mlx-community/Qwen3-4B-Instruct-2507-4bit
DATABASE_URL=sqlite:///./mlx_chat.db
JWT_SECRET_KEY=replace-this-secret-in-production
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
ALLOW_ORIGINS=*
  
```

Listing 5: Example backend environment configuration.

The backend can then be executed with:

```

cd mlx-openai-like-backend-main
uv sync
uv run uvicorn app.main:app --host 0.0.0.0 --port 8000
  
```

Listing 6: Starting the backend.

The API documentation should be available at <http://localhost:8000/docs>, and the health endpoint can be checked at <http://localhost:8000/health>.

7.2 Frontend Execution

The frontend can be configured and started as follows:

```

cd mlx-openai-like-frontend-main
export BACKEND_URL=http://localhost:8000
export FLASK_SECRET_KEY=replace-this-frontend-secret
uv sync
  
```

```
uv run python app/main.py
```

Listing 7: Starting the frontend.

After startup, the web interface should be accessible at <http://localhost:5001>. A functional test consists of creating an account, logging in, creating a chat, sending a prompt, testing both standard and streaming modes, and checking whether the chat history is persisted.

8 Discussion

8.1 Strengths

The prototype has several strengths. First, it separates user interface concerns from backend concerns and inference concerns. This improves maintainability and makes each layer easier to evolve. Second, the backend acts as a controlled gateway to the LLM server. This is important because it enables authentication, authorization, history persistence, and future governance capabilities. Third, the use of an OpenAI-compatible client with a configurable base URL reduces coupling to a specific inference provider. Fourth, the data model supports persistent chat history, enabling future features such as conversation search, summarization, session analytics, and audit logs. Fifth, streaming improves user experience by reducing perceived latency.

8.2 Limitations

The prototype also presents several limitations. The database configuration should be adjusted to avoid mixing a PostgreSQL default URL with SQLite-specific engine arguments. The backend should fully honor the model selected by the frontend. The streaming media type should be aligned with the SSE-like payload. Default secrets must be replaced before deployment. JWTs should not be printed in logs. The frontend should avoid rendering untrusted message content with unsafe HTML filters. The project also lacks automated tests, database migrations, structured logging, observability, rate limiting, and production-oriented deployment artifacts.

8.3 Security Considerations

Because conversational applications may process sensitive data, security is a central concern. The backend should enforce strict token validation, secure password hashing, HTTPS deployment, input validation, output sanitization, and least-privilege access to persisted chat histories. JWT secrets should be stored in a secure secret manager or environment-specific configuration. The frontend should avoid trusting generated model output as safe HTML because LLM responses can include arbitrary markup or scripts.

8.4 Future Evolution

A natural evolution is the inclusion of Retrieval-Augmented Generation. RAG combines a parametric language model with non-parametric memory, often represented by a vector index of retrieved documents [2]. This would allow the chatbot to

answer questions based on institutional documents, technical manuals, course materials, or project repositories.

Other future directions include Docker Compose deployment, PostgreSQL support with migrations, Prometheus metrics, Grafana dashboards, structured logs, role-based access control, prompt templates, multi-model routing, cost and token accounting, feedback collection, and empirical evaluation of latency, throughput, and response quality.

9 Threats to Validity

This study is based on technical inspection of the prototype artifacts rather than controlled experiments. Therefore, it does not report quantitative measurements of latency, throughput, memory consumption, token generation speed, or model answer quality. The behavior of the application may vary depending on the selected LLM, local hardware, inference server implementation, operating system, and Python environment. In addition, the OpenLike term used in this paper is an architectural label for OpenAI-compatible integration and not a formal standard. Despite these limitations, the inspection is sufficient to characterize the prototype architecture, describe its implementation, and identify relevant improvements.

10 Conclusion

This paper presented a prototype of an intelligent chatbot application integrated with LLMs through an OpenLike architecture. The solution combines a FastAPI backend, a Flask frontend, and an OpenAI-compatible LLM server. The backend provides JWT authentication, user management, chat session management, message persistence, model listing, health checks, and inference. The frontend provides registration, login, dashboard, chat, history, and support for both synchronous and streaming responses.

The prototype demonstrates a practical architecture for educational and experimental LLM applications. Its modular

organization supports maintainability and future extension. However, production use requires improvements in database configuration, secret management, streaming protocol consistency, HTML sanitization, logging, testing, and deployment automation. Future work should focus on RAG, observability, multi-model management, security hardening, empirical evaluation, and containerized deployment.

References

- [1] FastAPI. 2026. FastAPI Documentation. Online documentation. <https://fastapi.tiangolo.com/>
- [2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 9459–9474. <https://arxiv.org/abs/2005.11401>
- [3] MDN Web Docs. 2025. Using Server-Sent Events. Online documentation. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
- [4] OpenAI. 2026. The Official Python Library for the OpenAI API. GitHub repository. <https://github.com/openai/openai-python>
- [5] OWASP Foundation. 2026. Testing JSON Web Tokens. Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/10-Testing_JSON_Web_Tokens
- [6] Pallets Projects. 2026. Flask Documentation. Online documentation. <https://flask.palletsprojects.com/>
- [7] SQLAlchemy. 2026. SQLAlchemy 2.0 Documentation: ORM Quick Start and Declarative Mapping. Online documentation. <https://docs.sqlalchemy.org/orm/quickstart.html>
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates, Inc. <https://arxiv.org/abs/1706.03762>
- [9] WHATWG. 2026. HTML Living Standard: Server-Sent Events. Online specification. <https://html.spec.whatwg.org/multipage/server-sent-events.html>
- [10] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>